

1. Попередній аналіз даних

```
In [16]: # Імпорт бібліотек
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score,
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sqlalchemy import create_engine
from db_config import CONN_STR
import warnings
warnings.filterwarnings('ignore')

# Налаштування стилю графіків
plt.style.use('seaborn-v0_8')
%matplotlib inline

# Підключення до PostgreSQL
engine = create_engine(CONN_STR)

# SQL-запит для об'єднання таблиць
query = """
SELECT
    m.movie_id, m.title, m.year, m.genre, m.rating, m.runtime_minutes, m.
    bo.worldwide, bo.domestic, bo.domestic_percent, bo.foreign, bo.foreig
    mp.average_rating, mp.number_of_votes, mp.approval_index, mp.producti
    CASE WHEN oa.winner IS NOT NULL THEN 1 ELSE 0 END AS oscar_nominated
FROM main.dim_movie m
LEFT JOIN main.fact_box_office bo ON m.movie_id = bo.movie_id
LEFT JOIN main.fact_movie_performance mp ON m.movie_id = mp.movie_id
LEFT JOIN main.fact_oscar_awards oa ON m.movie_id = oa.movie_id
"""

# Завантаження даних
df = pd.read_sql(query, engine)

# Огляд структури даних
print("=== Огляд даних ===")
print(df.info())
print("\n=== Перші 5 рядків ===")
print(df.head())

# 1.1 Аналіз розподілу числових ознак
numeric_cols = ['budget', 'worldwide', 'domestic', 'foreign', 'runtime_mi
                'average_rating', 'number_of_votes', 'approval_index', 'p
                'domestic_gross', 'worldwide_gross']
plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(4, 3, i)
    sns.histplot(df[col].dropna(), bins=30, kde=True)
```

```
plt.title(f'Розподіл {col}')
plt.xlabel(col)
plt.ylabel('Частота')
plt.tight_layout()
plt.show()

# Boxplot для виявлення викидів
plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(4, 3, i)
    sns.boxplot(y=df[col].dropna())
    plt.title(f'Boxplot {col}')
plt.tight_layout()
plt.show()

# 1.1 Аналіз розподілу категоріальних ознак
categorical_cols = ['genre', 'rating', 'country', 'year']
plt.figure(figsize=(15, 10))
for i, col in enumerate(categorical_cols, 1):
    plt.subplot(2, 2, i)
    sns.countplot(y=df[col], order=df[col].value_counts().index[:10])
    plt.title(f'Розподіл {col}')
    plt.xlabel('Кількість')
    plt.ylabel(col)
plt.tight_layout()
plt.show()

# 1.2 Кореляційний аналіз
plt.figure(figsize=(12, 8))
corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Теплова карта кореляцій')
plt.show()

# Діаграми розсіювання для ключових пар
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
sns.scatterplot(x='budget', y='worldwide_gross', data=df)
plt.title('Бюджет vs Світові збори')
plt.subplot(1, 3, 2)
sns.scatterplot(x='average_rating', y='worldwide_gross', data=df)
plt.title('Рейтинг vs Світові збори')
plt.subplot(1, 3, 3)
sns.scatterplot(x='number_of_votes', y='worldwide_gross', data=df)
plt.title('Кількість голосів vs Світові збори')
plt.tight_layout()
plt.show()

# 1.3 Аналіз пропусків
print("\n=== Відсоток пропусків ===")
missing = df.isnull().mean() * 100
print(missing[missing > 0])

# Візуалізація пропусків
plt.figure(figsize=(10, 6))
sns.heatmap(df[numeric_cols].isnull(), cbar=False, cmap='viridis')
plt.title('Пропуски в числових ознаках')
plt.show()
```

=== Огляд даних ===

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9888 entries, 0 to 9887

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	movie_id	9888 non-null	int64
1	title	9888 non-null	object
2	year	9888 non-null	int64
3	genre	9888 non-null	object
4	rating	9811 non-null	object
5	runtime_minutes	9886 non-null	float64
6	budget	7591 non-null	float64
7	production_company	9873 non-null	object
8	country	9887 non-null	object
9	worldwide	3665 non-null	float64
10	domestic	3665 non-null	float64
11	domestic_percent	3665 non-null	float64
12	foreign	3665 non-null	float64
13	foreign_percent	3665 non-null	float64
14	average_rating	5330 non-null	float64
15	number_of_votes	5330 non-null	float64
16	approval_index	5330 non-null	float64
17	production_budget	5330 non-null	float64
18	domestic_gross	5330 non-null	float64
19	worldwide_gross	5330 non-null	float64
20	oscar_nominated	9888 non-null	int64

dtypes: float64(13), int64(3), object(5)

memory usage: 1.6+ MB

None

=== Перші 5 рядків ===

	movie_id	title	year
0	1055	Topsy-Turvy	1999
1	1033	The Time Machine	2002
2	918	The Way Back	2010
3	815	Ed Wood	1994
4	565	The Lord of the Rings: The Fellowship of the Ring	2001

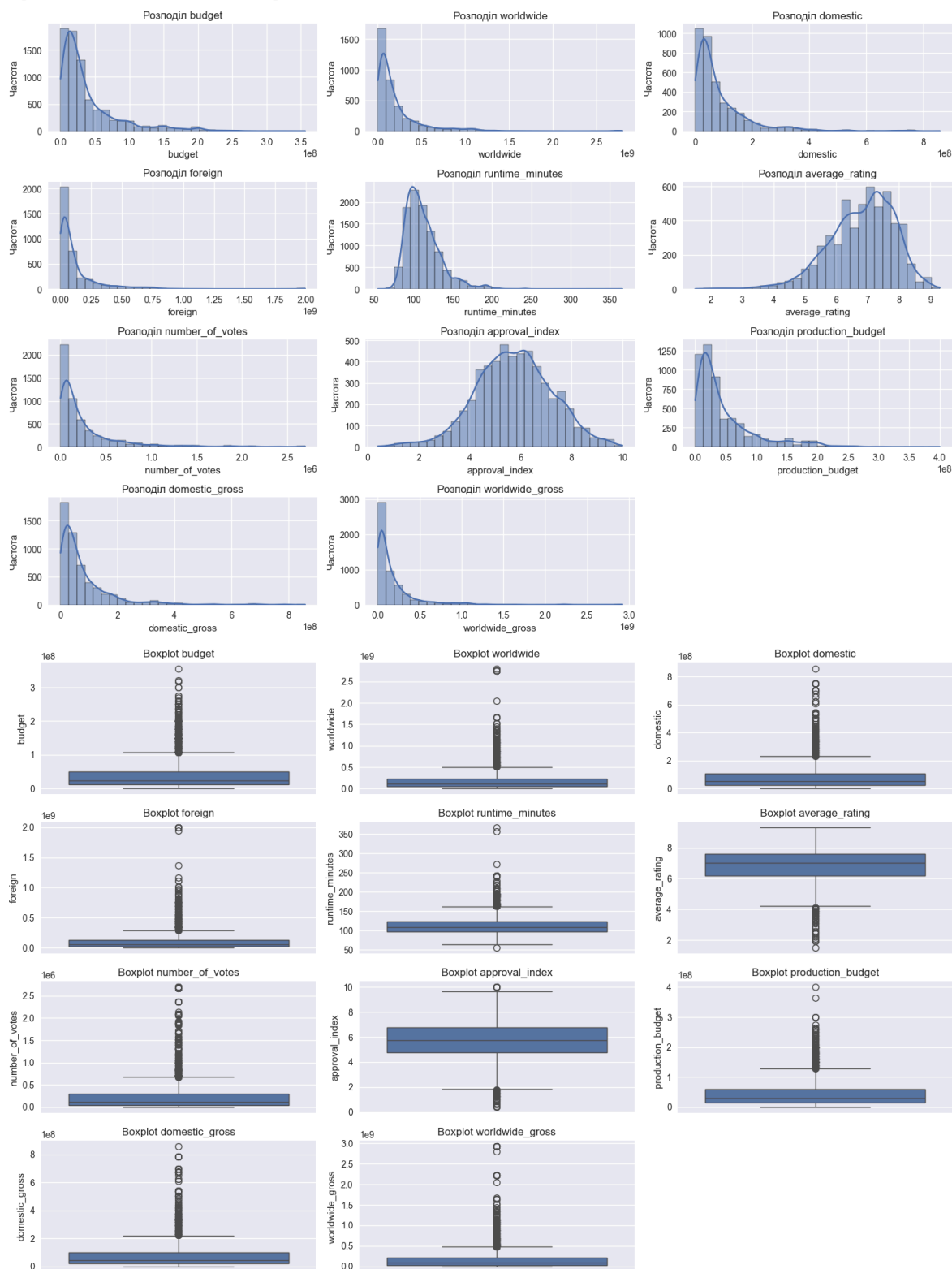
	genre	rating	runtime_minutes	budget	production_company
0	Biography	R	160.0	NaN	Goldwyn Films
1	Action	PG-13	96.0	80000000.0	Warner Bros.
2	Adventure	PG-13	133.0	30000000.0	Exclusive Films
3	Biography	R	127.0	18000000.0	Touchstone Pictures
4	Action	PG-13	178.0	93000000.0	New Line Cinema

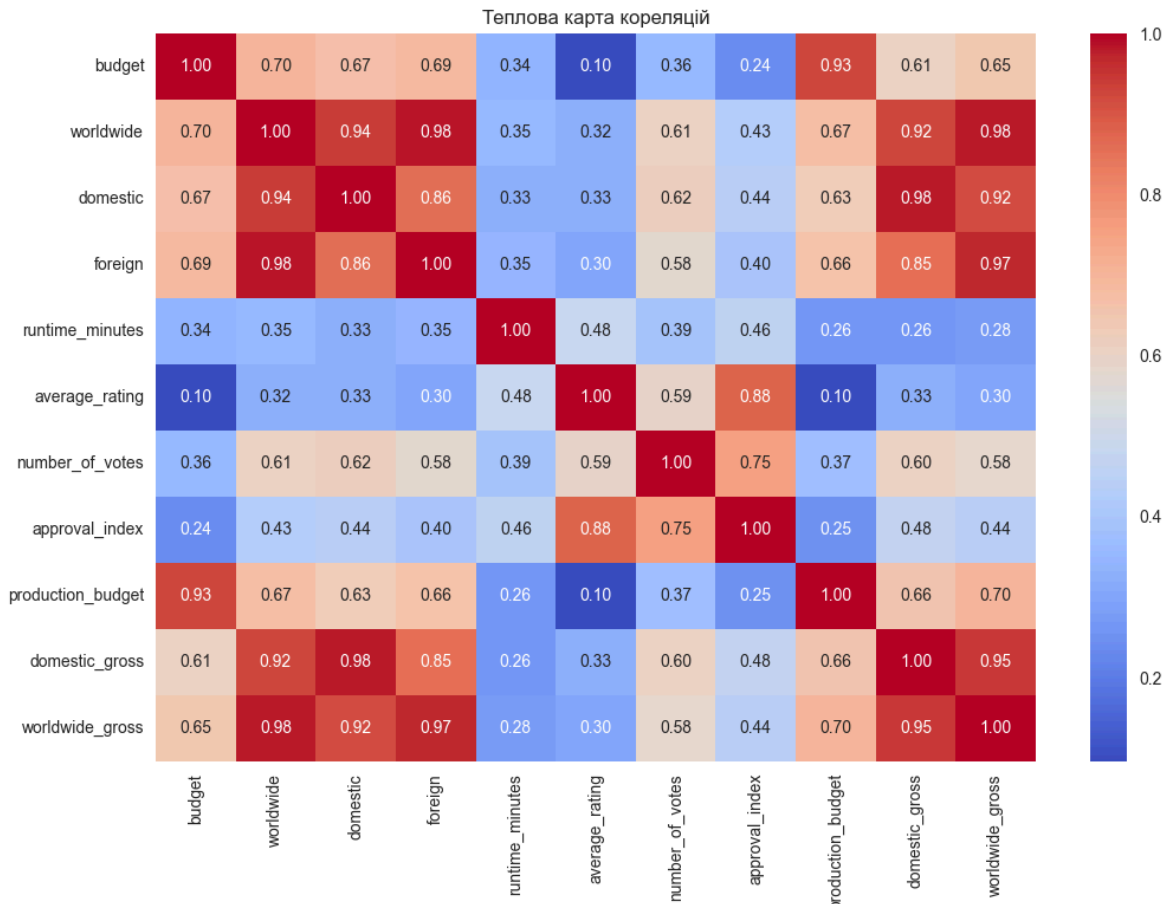
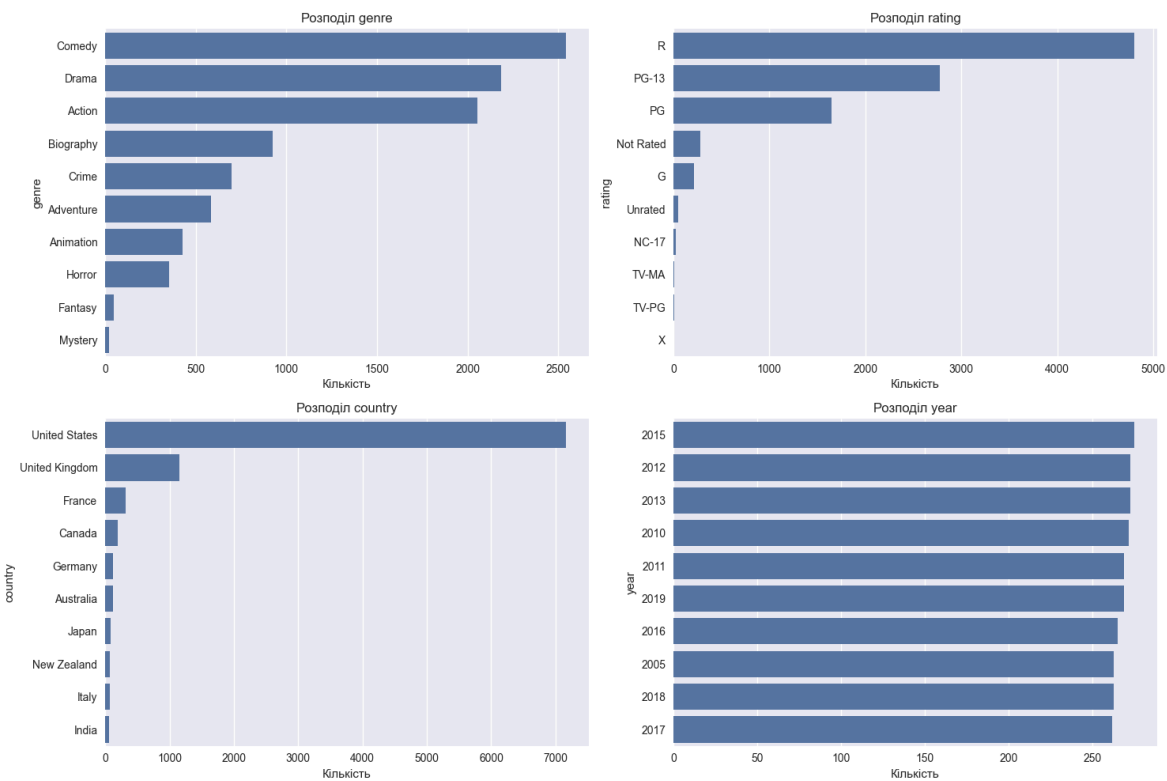
	country	worldwide	...	domestic_percent	foreign
0	United Kingdom	NaN	...	NaN	NaN
1	United States	123729176.0	...	0.46	66896682.0
2	United States	24172201.0	...	0.11	21470342.0
3	United States	NaN	...	NaN	NaN
4	New Zealand	868385360.0	...	0.36	555021246.0

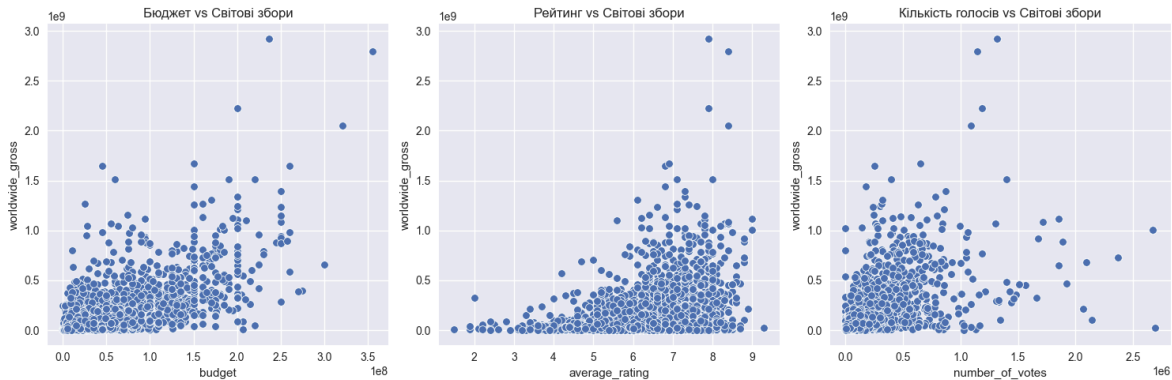
	foreign_percent	average_rating	number_of_votes	approval_index
0	NaN	NaN	NaN	NaN
1	0.54	6.0	126054.0	5.05
2	0.89	6.7	47905.0	5.18
3	NaN	7.8	178200.0	6.81
4	0.64	8.8	1886597.0	9.22

	production_budget	domestic_gross	worldwide_gross	oscar_nominated
0	NaN	NaN	NaN	1
1	80000000.0	56684819.0	98983590.0	1
2	23000000.0	13590514.0	14563409.0	1
3	18000000.0	5828466.0	5828734.0	1
4	93000000.0	315544750.0	891216824.0	1

[5 rows x 21 columns]

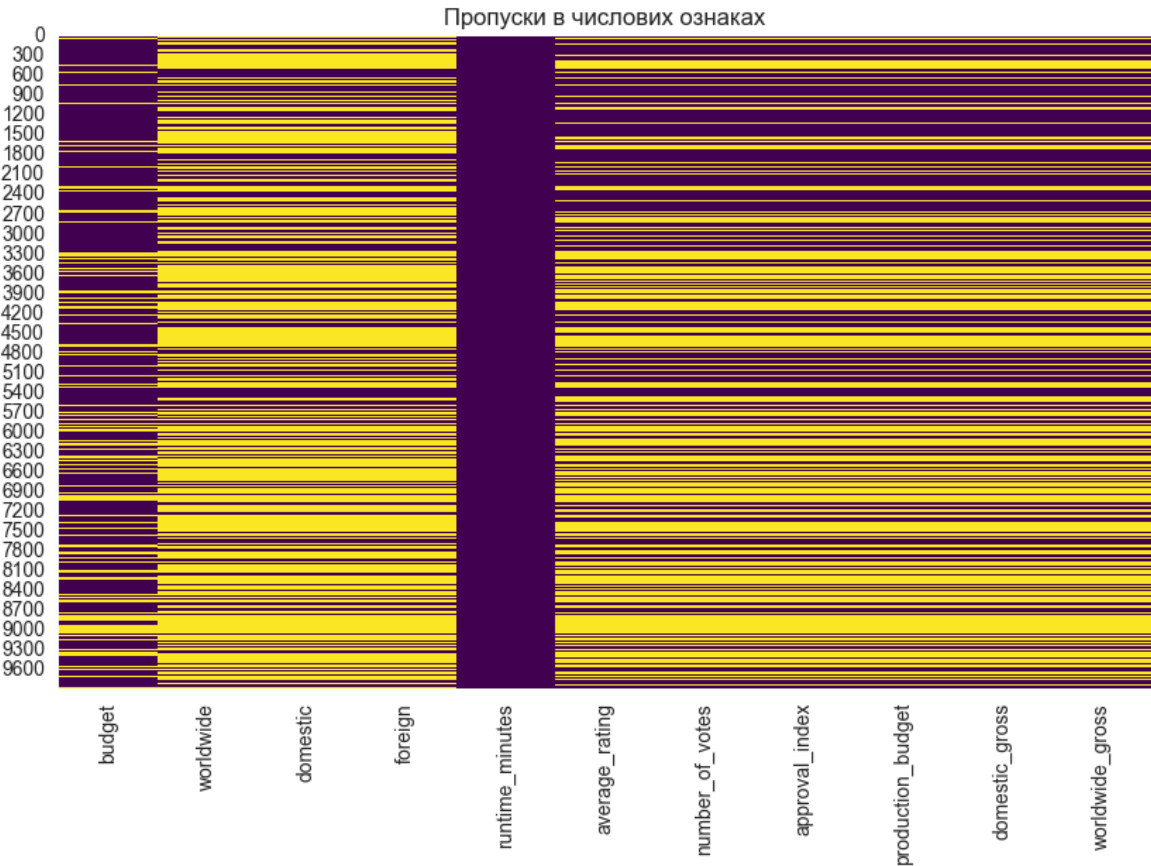






=== Відсоток пропусків ===

rating	0.778722
runtime_minutes	0.020227
budget	23.230178
production_company	0.151699
country	0.010113
worldwide	62.934871
domestic	62.934871
domestic_percent	62.934871
foreign	62.934871
foreign_percent	62.934871
average_rating	46.096278
number_of_votes	46.096278
approval_index	46.096278
production_budget	46.096278
domestic_gross	46.096278
worldwide_gross	46.096278
dtype:	float64



2. Підготовка даних

```
In [17]: # Припускаємо, що df уже завантажено
# 1. Обробка пропусків
# Категоріальні колонки
df['rating'].fillna('R', inplace=True)
df['production_company'].fillna('Unknown', inplace=True)
df['country'].fillna('United States', inplace=True)

# Числові колонки
df['runtime_minutes'].fillna(df['runtime_minutes'].median(), inplace=True)
df['budget'].fillna(df['budget'].median(), inplace=True)

# 2. Кодування категоріальних ознак
# One-hot encoding для genre та rating
df = pd.get_dummies(df, columns=['genre', 'rating'], prefix=['genre', 'ra

# Для country: топ-5 країн, решта – 'Other'
top_countries = ['United States', 'United Kingdom', 'France', 'Canada', '
df['country'] = df['country'].apply(lambda x: x if x in top_countries else
df = pd.get_dummies(df, columns=['country'], prefix='country')

# 3. Вибір ознак
features = ['year', 'runtime_minutes', 'budget'] + \
    [col for col in df.columns if col.startswith('genre_') or
     col.startswith('rating_') or col.startswith('country_')]

# 4. Дані для регресії
df_reg = df[df['worldwide_gross'].notnull()]
X_reg = df_reg[features]
y_reg = df_reg['worldwide_gross']
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

# 5. Дані для класифікації
X_class = df[features]
y_class = df['oscar_nominated']
X_train_class, X_test_class, y_train_class, y_test_class = train_test_spl
    X_class, y_class, test_size=0.2, random_state=42
)

print("Дані підготовлено для моделювання.")
```

Дані підготовлено для моделювання.

3. Моделювання

```
In [18]: # Масштабування числових ознак

scaler = StandardScaler()
numeric_features = ['year', 'runtime_minutes', 'budget']
X_train_reg[numeric_features] = scaler.fit_transform(X_train_reg[numeric_
X_test_reg[numeric_features] = scaler.transform(X_test_reg[numeric_featur
X_train_class[numeric_features] = scaler.fit_transform(X_train_class[nume
X_test_class[numeric_features] = scaler.transform(X_test_class[numeric_fe

# Функція для оцінки регресійних моделей
def evaluate_regression_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
```

```

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'MSE: {mse:.2f}, R²: {r2:.2f}')
return y_pred

# Функція для оцінки класифікаційних моделей
def evaluate_classification_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]
    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_pred_proba)
    print(f'Accuracy: {accuracy:.2f}, F1-score: {f1:.2f}, ROC-AUC: {roc_auc:.2f}')
    return y_pred, y_pred_proba

# Моделі для регресії
print('=== Регресія ===')
lin_reg = LinearRegression()
rf_reg = RandomForestRegressor(n_estimators=100, random_state=42)
xgb_reg = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100)

print('Лінійна регресія:')
y_pred_lin_reg = evaluate_regression_model(lin_reg, X_train_reg, y_train_reg)
print('Random Forest Regressor:')
y_pred_rf_reg = evaluate_regression_model(rf_reg, X_train_reg, y_train_reg)
print('XGBoost Regressor:')
y_pred_xgb_reg = evaluate_regression_model(xgb_reg, X_train_reg, y_train_reg)

# Моделі для класифікації
print('\n=== Класифікація ===')
log_reg = LogisticRegression(max_iter=1000)
rf_class = RandomForestClassifier(n_estimators=100, random_state=42)
xgb_class = xgb.XGBClassifier(objective='binary:logistic', n_estimators=100)

print('Логістична регресія:')
y_pred_log_reg, y_pred_proba_log_reg = evaluate_classification_model(log_reg, X_train_class, y_train_class, X_test_class, y_test_class)
print('Random Forest Classifier:')
y_pred_rf_class, y_pred_proba_rf_class = evaluate_classification_model(rf_class, X_train_class, y_train_class, X_test_class, y_test_class)
print('XGBoost Classifier:')
y_pred_xgb_class, y_pred_proba_xgb_class = evaluate_classification_model(xgb_class, X_train_class, y_train_class, X_test_class, y_test_class)

# Візуалізація для регресії
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.scatter(y_test_reg, y_pred_lin_reg, alpha=0.5)
plt.xlabel('Фактичні значення')
plt.ylabel('Прогнозовані значення')
plt.title('Лінійна регресія')
plt.subplot(1, 3, 2)
plt.scatter(y_test_reg, y_pred_rf_reg, alpha=0.5)
plt.xlabel('Фактичні значення')
plt.ylabel('Прогнозовані значення')
plt.title('Random Forest Regressor')
plt.subplot(1, 3, 3)
plt.scatter(y_test_reg, y_pred_xgb_reg, alpha=0.5)
plt.xlabel('Фактичні значення')
plt.ylabel('Прогнозовані значення')
plt.title('XGBoost Regressor')

```



```
plt.tight_layout()
plt.show()

# Візуалізація для класифікації (ROC-криві)
plt.figure(figsize=(10, 8))
fpr, tpr, _ = roc_curve(y_test_class, y_pred_proba_log_reg)
plt.plot(fpr, tpr, label='Логістична регресія')
fpr, tpr, _ = roc_curve(y_test_class, y_pred_proba_rf_class)
plt.plot(fpr, tpr, label='Random Forest Classifier')
fpr, tpr, _ = roc_curve(y_test_class, y_pred_proba_xgb_class)
plt.plot(fpr, tpr, label='XGBoost Classifier')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-криві')
plt.legend()
plt.show()

# Важливість ознак для Random Forest Regressor
importances = rf_reg.feature_importances_
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(12, 6))
plt.title('Важливість ознак (Random Forest Regressor)')
plt.bar(range(X_train_reg.shape[1]), importances[indices], align='center')
plt.xticks(range(X_train_reg.shape[1]), X_train_reg.columns[indices], rot=45)
plt.show()

# Важливість ознак для XGBoost Classifier
importances = xgb_class.feature_importances_
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(12, 6))
plt.title('Важливість ознак (XGBoost Classifier)')
plt.bar(range(X_train_class.shape[1]), importances[indices], align='center')
plt.xticks(range(X_train_class.shape[1]), X_train_class.columns[indices], rot=45)
plt.show()

# Матриці невідповідностей для класифікації
plt.figure(figsize=(15, 5))

# Матриця невідповідностей для Логістичної регресії
cm_log_reg = confusion_matrix(y_test_class, y_pred_log_reg)
plt.subplot(1, 3, 1)
sns.heatmap(cm_log_reg, annot=True, fmt='d', cmap='Blues')
plt.title('Матриця невідповідностей (Логістична регресія)')
plt.xlabel('Прогнозовані')
plt.ylabel('Фактичні')

# Матриця невідповідностей для Random Forest Classifier
cm_rf_class = confusion_matrix(y_test_class, y_pred_rf_class)
plt.subplot(1, 3, 2)
sns.heatmap(cm_rf_class, annot=True, fmt='d', cmap='Blues')
plt.title('Матриця невідповідностей (Random Forest Classifier)')
plt.xlabel('Прогнозовані')
plt.ylabel('Фактичні')

# Матриця невідповідностей для XGBoost Classifier
cm_xgb_class = confusion_matrix(y_test_class, y_pred_xgb_class)
plt.subplot(1, 3, 3)
sns.heatmap(cm_xgb_class, annot=True, fmt='d', cmap='Blues')
```

```
plt.title('Матриця невідповідностей (XGBoost Classifier)')
plt.xlabel('Прогнозовані')
plt.ylabel('Фактичні')

plt.tight_layout()
plt.show()
```

=== Регресія ===

Лінійна регресія:

MSE: 39731296871736072.00, R^2 : 0.45

Random Forest Regressor:

MSE: 15421971642688486.00, R^2 : 0.79

XGBoost Regressor:

MSE: 19117421903045784.00, R^2 : 0.74

=== Класифікація ===

Логістична регресія:

Accuracy: 0.79, F1-score: 0.64, ROC-AUC: 0.84

Random Forest Classifier:

Accuracy: 0.91, F1-score: 0.87, ROC-AUC: 0.95

XGBoost Classifier:

Accuracy: 0.88, F1-score: 0.82, ROC-AUC: 0.93



